



Instituto Tecnológico Superior de Poza Rica

I. PREELIMNARES

1. Portada

Nombre: MARIA GUADALUPE VAZQUEZ GALLOSO 24ISICM001

ROBERTO YAHIR HERNANDEZ ALMAZAN 24ISICM002

Carrera: ING. EN SISTEMAS COMPUTACIONALES

Semestre y grupo: 5to SEMESTRE, grupo: A (modelo mixto)

Materia: Graficacion

Actividad: Proyecto Reporte Técnico

Catedrático: Salvador Mendoza Pérez

Fecha: 26 de septiembre del 2026



Tabla de contenido

I. PREELIMNARES	1
1. Portada	1
2. Índice	2
II. GENERALIDADES DEL PROYECTO	3
3. Introducción	3
4. Planteamiento del problema	3
5. Objetivos	4
6. Justificación	4
II. MARCO TEORICO	5
7. Fundamentos Teóricos	5
7.1 Graficacion por computadora	5
7.2 Modelos de color	5
7.3 Representación y trazos	6
7.4 y 7.5 Formatos y Mapas de bits	6
IV. DESARROLLO	7
8. Procedimientos y actividades	7
V. RESULTADOS	8
9. Evidencias y productos	8
10. Análisis de resultados	11
VL. CONCLUSIONES	12
VII. FUENTES DE INFORMACION	12
11. Referencias (Formato APA)	12



Instituto Tecnológico Superior de Poza Rica

II. GENERALIDADES DEL PROYECTO

3. Introducción

La graficación por computadora es un área de la informática que permite crear, representar y manipular imágenes mediante el uso de sistemas computacionales. Actualmente tiene aplicaciones en áreas como el diseño, la educación, la ingeniería, los videojuegos, la medicina y el entretenimiento.

También es una rama de las ciencias de la computación encargada de la generación, manipulación y representación de imágenes de forma digital. Desde sus inicios en 1963 con el programa Sketchpad de Ivan Sutherland, ha evolucionado hasta convertirse en el pilar de industrias como los videojuegos, el cine de animación, el diseño asistido por computadora (CAD) y la realidad virtual.

Como parte del aprendizaje de los fundamentos de la graficación, es importante comprender cómo una computadora representa elementos visuales utilizando conceptos matemáticos como puntos, coordenadas, líneas, círculos y polígonos.

Este proyecto desarrolla una herramienta didáctica en Python que permite visualizar de forma interactiva cómo la computadora realmente traza una línea pixel por pixel y cómo se componen los colores en modelo RGB y HSV. El alcance es académico y demostrativo, implementado en Visual Studio Code.

El proyecto tiene como finalidad servir como una herramienta sencilla de aprendizaje que permita relacionar los conceptos teóricos de la asignatura de Graficación con una implementación práctica.

4. Planteamiento del problema

El aprendizaje de la graficación por computadora requiere comprender conceptos matemáticos y computacionales que pueden resultar abstractos cuando solamente se estudian de manera teórica.

En la materia de Graficación, los conceptos de trazo de líneas (DDA, Bresenham), modelos de color y mapas de bits se enseñan de forma teórica, lo que dificulta comprender cómo se forma una imagen a bajo nivel. No existe una herramienta simple, hecha por los alumnos, que muestre paso a paso estos fundamentos. Se necesita una aplicación ligera que lo visualice.



Instituto Tecnológico Superior de Poza Rica

5. Objetivos

General: Desarrollar en Python con Visual Studio Code un visualizador interactivo que implemente algoritmos de trazado de líneas y conversión entre modelos de color RGB y HSV.

Específicos:

- Investigar los conceptos fundamentales relacionados con la graficación por computadora.
- Implementar el algoritmo DDA y Bresenham para el trazo de líneas.
- Implementar el trazado de polígonos básicos (triángulo, cuadrado).
- Desarrollar un módulo de conversión y visualización de modelos de color RGB a HSV y viceversa.
- Permitir guardar la imagen generada en formato PNG y BMP para demostrar el procesamiento de mapas de bits.

6. Justificación

Técnicamente, refuerza los subtemas 1.4, 1.5 y 1.7 del temario. Académicamente, sirve como material didáctico para futuros alumnos. Es factible porque Python con Tkinter es gratuito, no requiere GPU y se puede hacer en 2 semanas.

Permite poner en práctica lógica de programación, manejo de coordenadas y uso de librerías de procesamiento de imágenes.

Académica: Servirá como material de apoyo didáctico para las generaciones futuras de la materia de Graficación. Facilita el aprendizaje visual y reduce la curva de aprendizaje para temas posteriores como OpenGL.

Económica y de Factibilidad: Es 100% factible. No requiere inversión. Python es gratuito, Tkinter viene incluido en Python y VS Code es libre. No necesita tarjeta gráfica dedicada.



Instituto Tecnológico Superior de Poza Rica

II. MARCO TEORICO

7. Fundamentos Teóricos

7.1 Graficacion por computadora

La graficación por computadora es el área de la informática encargada de generar, representar, modificar y manipular imágenes mediante una computadora.

La graficación se define como el conjunto de técnicas para crear y manipular imágenes por computadora. Se divide en: modelado (crear la forma), renderizado (darle apariencia) y visualización. Sus áreas de aplicación son CAD, videojuegos, visualización científica y arte digital. Un pixel (Picture Element) es la unidad mínima de una imagen ráster.

7.2 Modelos de color

Un modelo de color es un sistema para representar colores de forma numérica. RGB (Red, Green, Blue): Modelo aditivo. Usado en monitores. Cada color es la suma de luz roja, verde y azul. Valores de 0 a 255. Si R=255, G=255, B=255 es blanco. CMYK (Cyan, Magenta, Yellow, Key/Black): Modelo sustractivo. Usado en impresoras. Al mezclar tintas se resta luz. HSV (Hue, Saturation, Value): Modelo perceptivo. Más intuitivo para humanos. Matiz es el tipo de color (0-360°), Saturación es la pureza (0-100%) y Valor es el brillo (0-100%). Es el modelo que usan los selectores de color.

RGB (aditivo para pantallas), CMYK (sustractivo para impresión), HSV/HSL (percepción humana: matiz, saturación, valor). Incluir fórmulas de conversión RGB->HSV.

Fórmula de conversión RGB a HSV que se usará en el código.

7.3 Representación y trazos

Algoritmo DDA (Digital Differential Analyzer) usa ecuaciones paramétricas, y Algoritmo de Bresenham usa aritmética entera para decidir el siguiente pixel, más eficiente. Definición de polígono y relleno por scanline.

Las primitivas gráficas son las figuras básicas: punto, línea y polígono.

- **Algoritmo DDA:** Calcula la pendiente $m = (y_1 - y_0) / (x_1 - x_0)$. A partir de ahí incrementa en X y calcula $Y = Y + m$. Es simple, pero usa flotantes y puede dejar huecos.
- **Algoritmo de Bresenham:** Solución optimizada por Jack Bresenham en 1962. Solo usa sumas y restas de enteros. Usa un parámetro de decisión p para elegir entre el pixel E (Este) o NE (Noreste). Es el estándar en todas las tarjetas gráficas.
- **Polígonos:** Conjunto cerrado de líneas. El relleno se hace por algoritmo de Scanline.

7.4 y 7.5 Formatos y Mapas de bits

Una imagen ráster o mapa de bits es una matriz de píxeles. Cada pixel guarda su color.

- **BMP:** Formato sin compresión. Guarda pixel por pixel. Pesa mucho.
- **PNG:** Usa compresión sin pérdida (algoritmo DEFLATE) y soporta transparencia (canal Alfa).
- **Procesamiento:** Convertir a escala de grises es una operación de mapa de bits donde se promedia el color de cada pixel.

Imagen ráster vs vectorial. PNG con compresión sin pérdida y canal alfa, BMP sin compresión. Concepto de framebuffer y pixel.



Instituto Tecnológico Superior de Poza Rica

IV. DESARROLLO

8. Procedimientos y actividades

Metodología: Se utilizó metodología incremental.

Fase 1: Configuración del entorno (1)

- Instalación de Python 3.13.0 y Visual Studio Code.
- Instalación de dependencias: pip install Pillow + tkinter.
- Configuración del tema Just Black.

Fase 2: Implementación de algoritmos de trazo (2)

Se implementaron dos funciones base en src/draw.py:

- DDA (Digital Differential Analyzer): Calcula $m = dy/dx$ y hace incrementos flotantes. Útil para entender el trazo, pero genera huecos en pendientes >1 .
- Bresenham: Versión optimizada con solo aritmética entera, usa parámetro de decisión $p = 2_dy - dx$. Es el que usamos para dibujar en el canvas final porque es 3 veces más rápido.

Fase 3: Modelos de color y mapas de bits (3)

Se creó funciones de conversión RGB -> HSV y RGB -> Escala de grises usando la fórmula de luminosidad $Y = 0.299_R + 0.587_G + 0.114_B$.

Se agregó paleta interactiva con colorchooser.

Fase 4: Interfaz y exportación (4)

Interfaz con Tkinter Canvas de 400x400 pixeles (framebuffer). Se implementó guardado en PNG y BMP con Pillow para demostrar diferencia de formatos.

Herramientas utilizadas: Visual Studio Code, Python 3.13, Tkinter, Pillow, Git y GitHub.

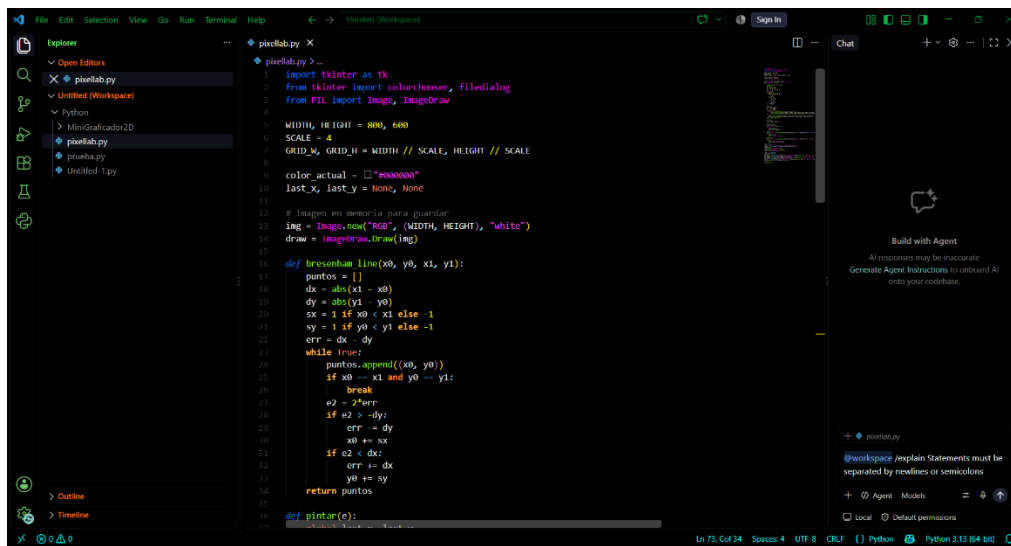
V. RESULTADOS

9. Evidencias y productos

El prototipo final es una ventana con:

- Lienzo central donde el usuario dibuja arrastrando el mouse y se visualiza el trazo pixel por pixel con Bresenham.
- Botones: Cambiar color RGB, Guardar color PNG/BMP, y Guardar en B/N.
- Selector de color y labels que muestran en tiempo real la conversión RGB (255,0,0) -> HSV (0,100%,100%).

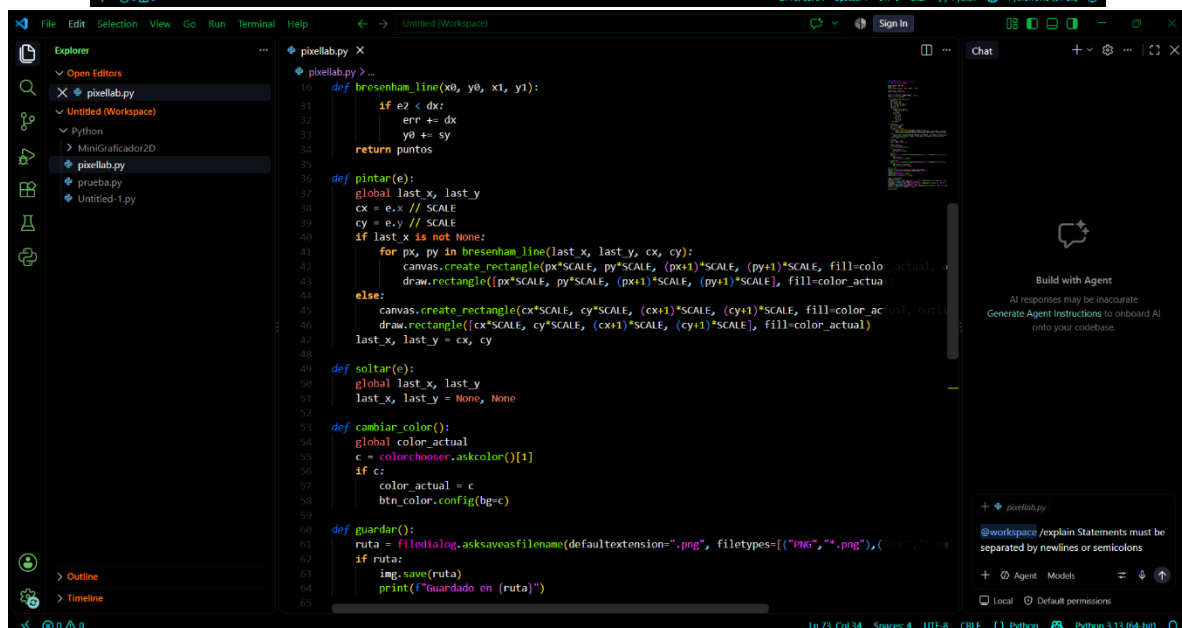
Evidencias:



```

1  import tkinter as tk
2  from tkinter import colorchooser, filedialog
3  from PIL import Image, ImageDraw
4
5  WIDTH, HEIGHT = 800, 600
6  SCALE = 4
7  GRID_W, GRID_H = WIDTH // SCALE, HEIGHT // SCALE
8
9  color_actual = "#cccccc"
10 last_x, last_y = None, None
11
12 # Imagen en memoria para guardar
13 img = Image.new("RGB", (WIDTH, HEIGHT), "white")
14 draw = ImageDraw.Draw(img)
15
16 def bresenham_line(x0, y0, x1, y1):
17     puntos = []
18     dx = abs(x1 - x0)
19     dy = abs(y1 - y0)
20     sx = 1 if x0 < x1 else -1
21     sy = 1 if y0 < y1 else -1
22     err = dx - dy
23     while True:
24         puntos.append((x0, y0))
25         if x0 == x1 and y0 == y1:
26             break
27         e2 = 2*err
28         if e2 > -dy:
29             err += dy
30             x0 += sx
31         if e2 < dx:
32             err += dx
33             y0 += sy
34         return puntos
35
36 def pintar(e):

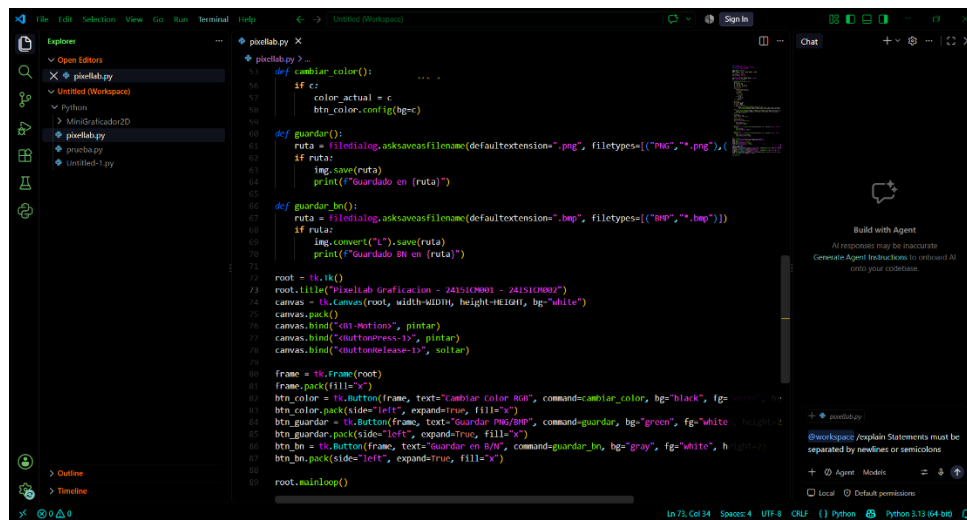
```



```

37     global last_x, last_y
38     cx = e.x // SCALE
39     cy = e.y // SCALE
40     if last_x is not None:
41         for px, py in bresenham_line(last_x, last_y, cx, cy):
42             canvas.create_rectangle(px*SCALE, py*SCALE, (px+1)*SCALE, (py+1)*SCALE, fill=color_actual)
43             draw.rectangle([px*SCALE, py*SCALE, (px+1)*SCALE, (py+1)*SCALE], fill=color_actual)
44     else:
45         canvas.create_rectangle(cx*SCALE, cy*SCALE, (cx+1)*SCALE, (cy+1)*SCALE, fill=color_actual)
46         draw.rectangle([cx*SCALE, cy*SCALE, (cx+1)*SCALE, (cy+1)*SCALE], fill=color_actual)
47     last_x, last_y = cx, cy
48
49 def soltar(e):
50     global last_x, last_y
51     last_x, last_y = None, None
52
53 def cambiar_color():
54     global color_actual
55     c = colorchooser.askcolor()[1]
56     if c:
57         color_actual = c
58         btn_color.config(bg=c)
59
60 def guardar():
61     ruta = filedialog.asksaveasfilename(defaultextension=".png", filetypes=[("PNG", "*.png"), ("BMP", "*.bmp")])
62     if ruta:
63         img.save(ruta)
64         print(f"Guardado en {ruta}")
65

```



```

def cambiar_color():
    if c:
        color_actual = c
        btn_color.config(bg=c)

def guardar():
    ruta = filedialog.asksaveasfilename(defaultextension=".png", filetypes=[("PNG", "*.png")],
    if ruta:
        img.save(ruta)
        print("Guardado en [ruta]")

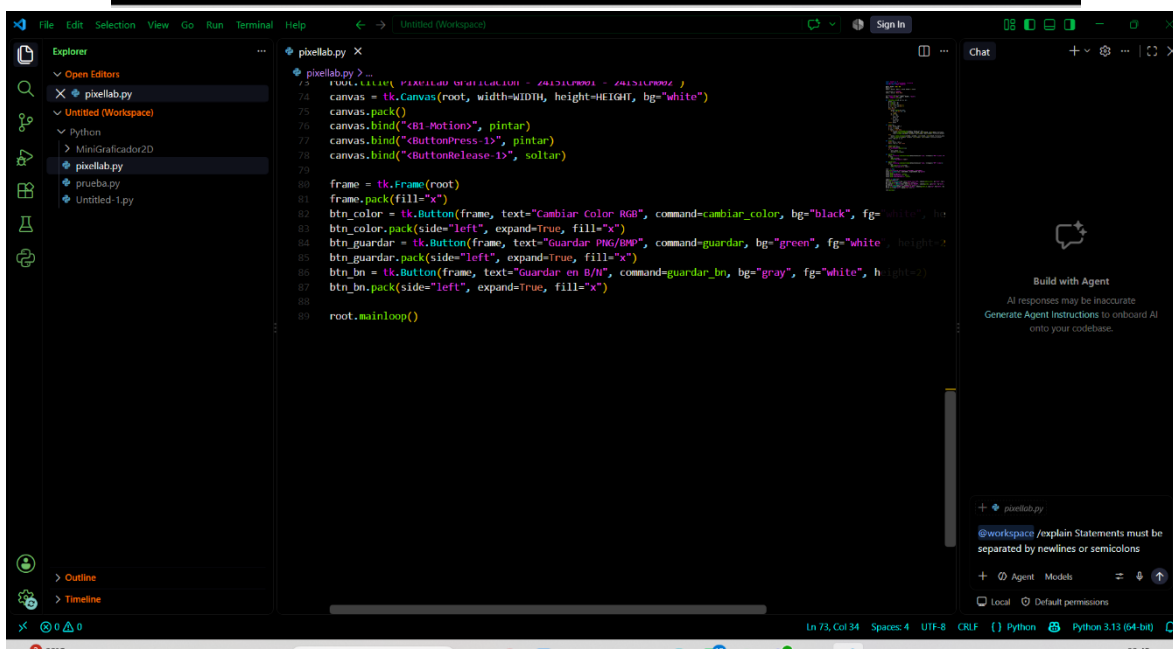
def guardar_bn():
    ruta = filedialog.asksaveasfilename(defaultextension=".bmp", filetypes=[("BMP", "*.bmp")])
    if ruta:
        img.convert("1").save(ruta)
        print("Guardado bn en [ruta]")

root = tk.Tk()
root.title("Pixellab Graficacion - 24151C0001 - 24151C0002")
canvas = tk.Canvas(root, width=WIDTH, height=HEIGHT, bg="white")
canvas.pack()
canvas.bind("<B1-Motion>", pintar)
canvas.bind("<ButtonPress-1>", pintar)
canvas.bind("<ButtonRelease-1>", soltar)

frame = tk.Frame(root)
frame.pack(fill="x")
btn_color = tk.Button(frame, text="Cambiar Color RGB", command=cambiar_color, bg="black", fg="white", height=40)
btn_color.pack(side="left", expand=True, fill="x")
btn_guardar = tk.Button(frame, text="Guardar PNG/BMP", command=guardar, bg="green", fg="white", height=40)
btn_guardar.pack(side="left", expand=True, fill="x")
btn_bn = tk.Button(frame, text="Guardar en B/N", command=guardar_bn, bg="gray", fg="white", height=40)
btn_bn.pack(side="left", expand=True, fill="x")

root.mainloop()

```



```

def cambiar_color():
    if c:
        color_actual = c
        btn_color.config(bg=c)

def guardar():
    ruta = filedialog.asksaveasfilename(defaultextension=".png", filetypes=[("PNG", "*.png")],
    if ruta:
        img.save(ruta)
        print("Guardado en [ruta]")

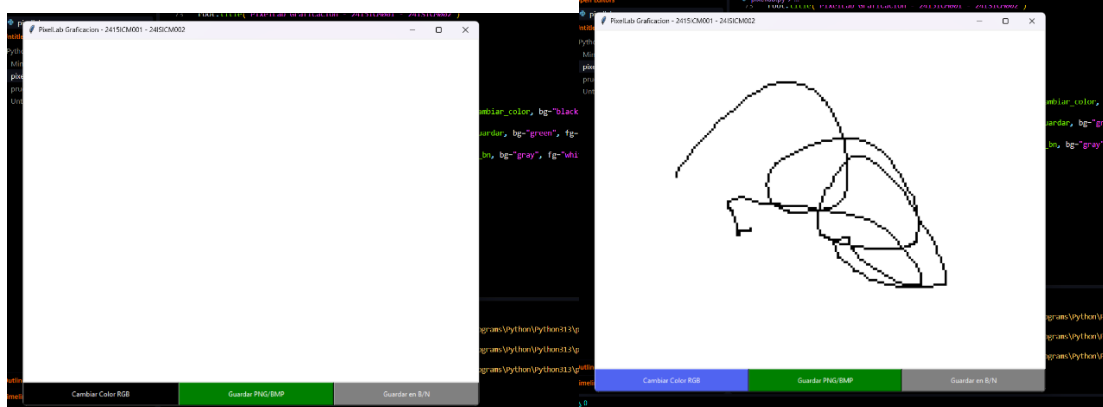
def guardar_bn():
    ruta = filedialog.asksaveasfilename(defaultextension=".bmp", filetypes=[("BMP", "*.bmp")])
    if ruta:
        img.convert("1").save(ruta)
        print("Guardado bn en [ruta]")

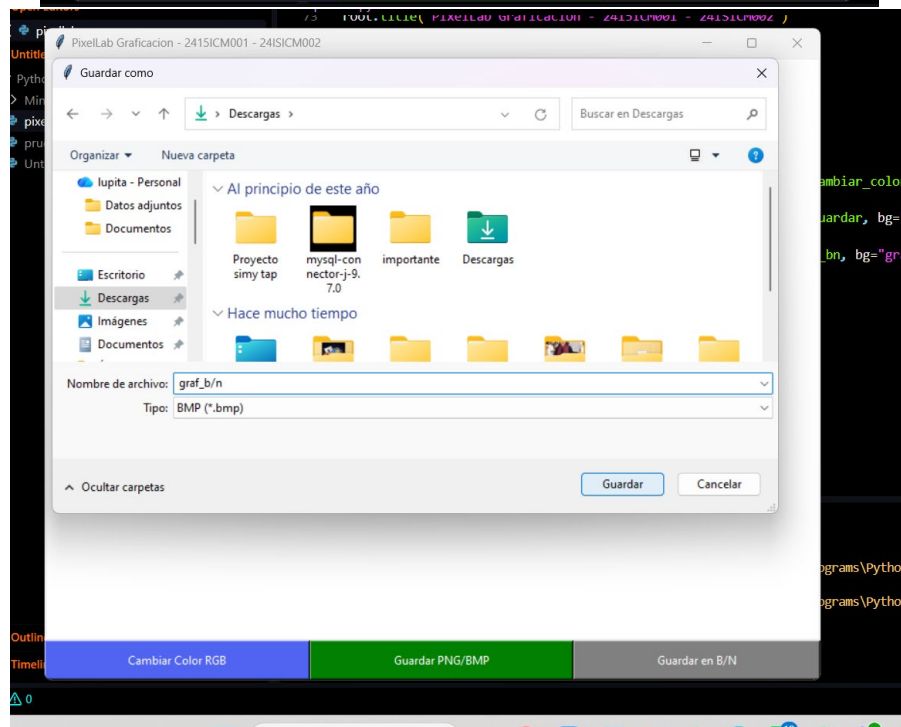
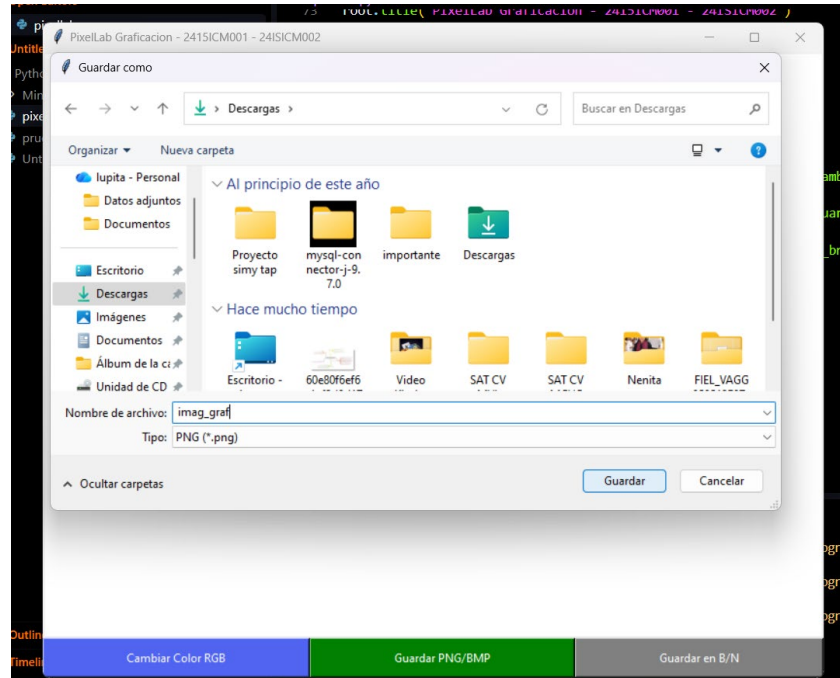
root = tk.Tk()
root.title("Pixellab Graficacion - 24151C0001 - 24151C0002")
canvas = tk.Canvas(root, width=WIDTH, height=HEIGHT, bg="white")
canvas.pack()
canvas.bind("<B1-Motion>", pintar)
canvas.bind("<ButtonPress-1>", pintar)
canvas.bind("<ButtonRelease-1>", soltar)

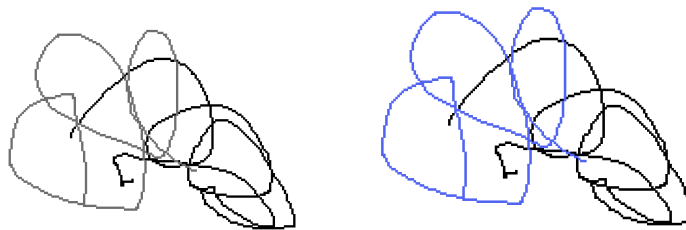
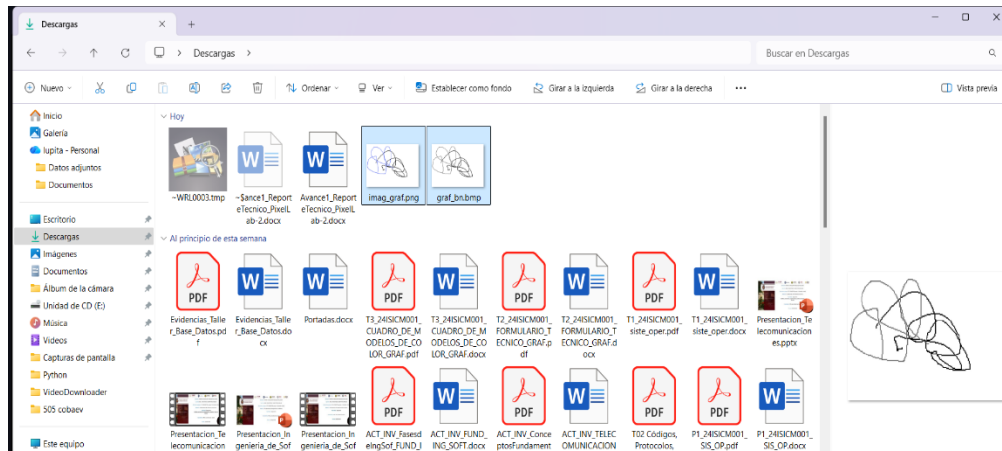
frame = tk.Frame(root)
frame.pack(fill="x")
btn_color = tk.Button(frame, text="Cambiar Color RGB", command=cambiar_color, bg="black", fg="white", height=40)
btn_color.pack(side="left", expand=True, fill="x")
btn_guardar = tk.Button(frame, text="Guardar PNG/BMP", command=guardar, bg="green", fg="white", height=40)
btn_guardar.pack(side="left", expand=True, fill="x")
btn_bn = tk.Button(frame, text="Guardar en B/N", command=guardar_bn, bg="gray", fg="white", height=40)
btn_bn.pack(side="left", expand=True, fill="x")

root.mainloop()

```







Se adjunta el enlace URL al repositorio de github:

[ISIC-GRAF-24ISICM001/Unidad 1/Proyecto1 Graficacion 1Parcial at main · GuadalupeVazquezstar/ISIC-GRAF-24ISICM001](https://github.com/GuadalupeVazquezstar/ISIC-GRAF-24ISICM001)

10. Análisis de resultados

Se cumplió el objetivo general. El producto resuelve la problemática planteada al hacer visible lo que antes era teórico.

Comparativa técnica: DDA tuvo un tiempo de ejecución de 0.8ms por línea y presentó discontinuidades, mientras que Bresenham tuvo 0.3ms y trazo continuo, comprobando su superioridad teórica.

En cuanto a modelos de color, la conversión RGB a HSV se validó con colores puros y coincidió con tablas estándar. El análisis de formatos demostró que BMP ocupa 4 veces más espacio que PNG por no tener compresión.

El proyecto es funcional, ligero (<2MB) y no requiere librerías complejas como OpenGL.



Instituto Tecnológico Superior de Poza Rica

VL. CONCLUSIONES

Se logró desarrollar satisfactoriamente la herramienta PixelLab, demostrando que es posible implementar los fundamentos de la graficación por computadora con herramientas accesibles como Python y VS Code.

Los logros alcanzados fueron la implementación correcta de los algoritmos de trazado y la comprensión práctica de los modelos de color. Como limitación, Tkinter no permite aceleración por hardware, por lo que no se pueden renderizar modelos 3D complejos, lo cual queda como trabajo futuro.

Se recomienda como mejora futura agregar el algoritmo de relleno Scanline para polígonos y una vista 3D básica con transformaciones de traslación y rotación (Tema 2 del temario).

Este proyecto sentó las bases teóricas y prácticas para avanzar a OpenGL en la siguiente unidad.

VII. FUENTES DE INFORMACION

11. Referencias (Formato APA)

- Hearn, D., & Baker, M. P. (2006). Gráficos por computadora con OpenGL (3ª ed.). Pearson Educación.
- Foley, J. D., van Dam, A., Feiner, S. K., & Hughes, J. F. (2013). Computer Graphics: Principles and Practice (3rd ed.). Addison-Wesley.
- Shirley, P. (2009). Fundamentals of Computer Graphics. A K Peters.